

Build A Model

03/25/2025 17:02 and 03/26/2025 16:04

Design a class that encapsulates an ArrayList of Players. This class will allow a client to add a player, find out the name or score of a player (by giving the index of that Player in the list) and set the score of a player (by providing the index of that player and the new score value)

I gave it access to no other files

The AI did a great job at this task. There was no need for any modifications at first when I was just testing this step. However when I came to the final step I noticed that the AI did not quite create the parameters correctly. I had to go back and make sure the ArrayList was of type Player.

Build A Viewer

03/25/2025 17:08

Build a new class that encapsulates the idea of a view for the model (scoreboard). This should work similar to BarViewer or A TextViewer classes. It needs to show multiple items (one name and one score per Player).

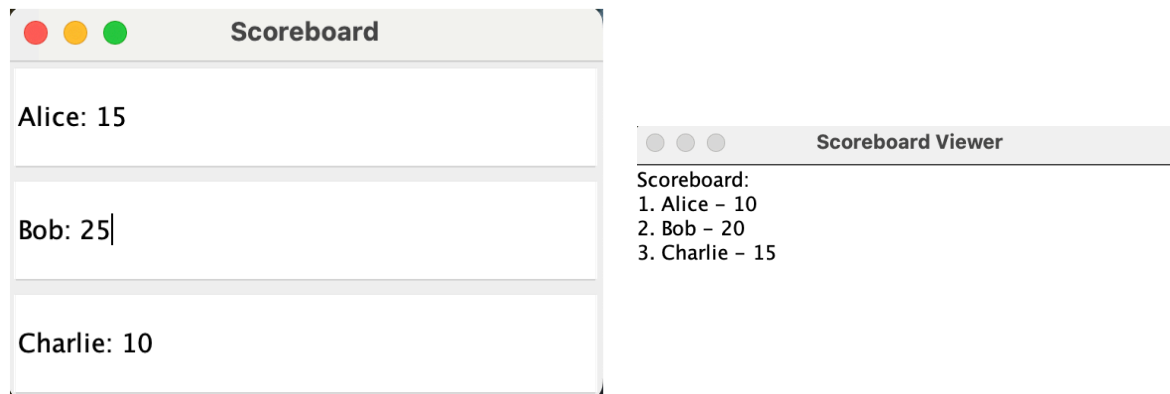
I gave it access to no other files

Once again the AI did a great job with this step. For the first step of this process I did not have to do any modifications myself. Similar to the step above, I had to go back and make sure that the ArrayList was of type Player.

03/26/2025 16:06

Build a new class that encapsulates the idea of a view for the model (scoreboard). This should work similar to the TextViewer class provided. The information it needs to show is the players names and their current score.

When I asked the AI to perform this step it created a slightly different scoreboard. The first scoreboard created a TextViewer for each player. Whereas when prompted again after my code was deleted it had all the information in one TextViewer box. The first image is the score board created for the first time. The second image is the scoreboard created the second time.



03/25/2025 17:12 and 03/26/2025 16:09

have it create a main method that populates the a "game roster" with both names and scores and then display their values

I gave it access to the ScoreboardView.Java file the AI created.

At first when prompted the AI wanted to create a whole new class called main. I can see why the AI would think to do this. Based on my last two prompts I asked it to create new classes. I assume that the AI picked up on this "pattern" and thought to do the same. This was a simple manual fix of just copying and pasting the code generated in the main class and placing it in the ScoreBoardView.Java file.

After my code was deleted the AI encountered an issue where it kept trying to make the dummy players of type player. I tried prompting it again to make it of type WimpPlayer but keep trying reverting back to making it of type Player. So I had to manually make it of type WimpPlayer.

03/25/2025 17:20 and 03/26/2025 16:18

our main program should then put up an OK-only JOptionPane dialog (to cause a pause) before changing the score of a Player. If all works as it should, clicking OK in the dialog will cause the Model to change, triggering a (visible) change in the Viewer/Scoreboard.

I gave it access to the ScoreboardView.Java file the AI created.

For this prompt I would state that the AI did an adequate job. It did create the pane with the dialog, however what it did not do was have it update the score. This was fixed in a future prompt.

Postcode deletion was able to have the score updated properly after this prompt. Thus the future prompt was not needed.

03/25/2025 17:23 and 03/26/2025 16:20

have the display be based off of the TextViewer class. Make sure it shows one name and one score per player

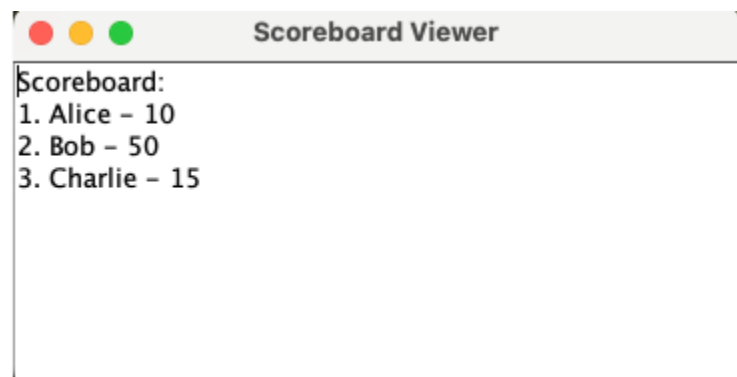
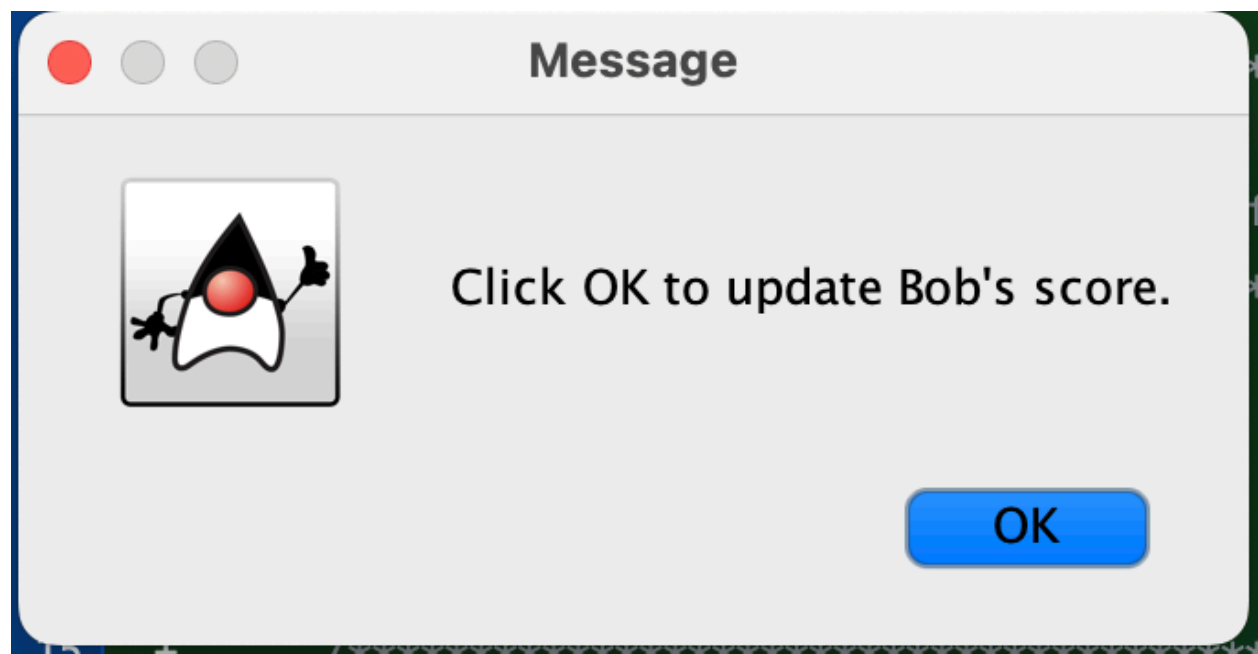
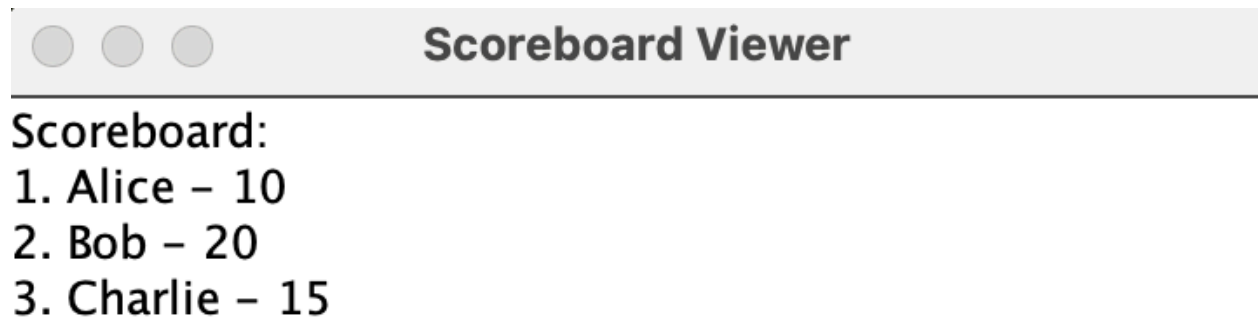
I gave it access to the TextViewer Classes and ScoreboardView.java

The AI was able to execute this step with no errors. It did a good job at replicating the format provided and updating it to match what was needed.

03/25/2025 17:25

When clicking OK it should cause the model to change updating the score
I gave it access to ScoreboardView.java

The AI did a good job at what it was asked to do to update the score.



Integration Into Bulldog

03/25/2025 17:33 and 03/26/2025 16:27

Change Bulldog to use your Model rather than a specific ArrayList of Players.

I gave it access to ScoreboardView.java and PlayerList.java

I wanted to give the AI a bit of a challenge to use as little of the code that you provided us with as possible. The AI passed with flying colors. It was able to create a Model and ModelObserver easily.

The second time this was done the AI didn't need to create a model and ModelObserver class it was able to put the information store there previously in other places. The switch here was simpler and much more efficient.

03/25/2025 17:40

Intergrate the scoreboard into the Bulldog GUI

I gave it access to ScoreboardView.java

During the integration step the AI struggled to let go of the preset list of players it created during one of the earlier steps. When you start to play the first game it wants to play with the list of players the AI created. However the Play Again worked and allowed the game to be played properly with the set of players the user dictated. This was something I had to go back and manually change. Another thing I had to manually do was have it display the scoreboard at the same time the PlayScreen was active.

03/26/2025 16:35

Integrate the scoreboard into the playscreen.java. Have it show it beside it

I gave it access to home.java

This was performed postcode deletion. This was one of the prompts I modified to help with integration. When asked this prompt I did not have to do any manual modifications.

Time Management

Before Code Deletion: The time it took for the AI to write and generate code was around 45 minutes. For my manual edits I had to do with the code I spent around 15 minutes. I spent around 50 minutes debugging and testing to ensure that the program worked as intended. I spent an additional hour writing the report. This all came to a grand total of roughly 2 hours and 50 minutes.

After Code Deletion: The time it took for the AI to write the code again after my code was deleted was less. It took 40 minutes. This is largely due to the fact that I did not have to type y prompts into the AI I was able to just copy and paste. My manual edits took 10 minutes this time. I believe this is largely due to the fact that I knew how to prompt the AI better and was able to give it better guidance. Debugging and testing took around 15 minutes this time. I knew what to look for. To edit the lab report to include the new AI prompts and responses took me around 20 minutes. As a whole after the deletion of my code it took significantly less time. This is due to the fact that most of the report was already written and I had a better understanding of how to prompt the AI better.

Reflection

I honestly struggled with this assignment as a whole. I had trouble understanding the instructions. This made it difficult for me to check if things were correct and find bugs and errors. Although the AI was able to write a large amount of the code. I still need to understand how this should function. In the end I got there even though I found it difficult.

The biggest challenge overall for me was having to essentially redo the entire assignment. I was very fortunate to have the same prompts I used before to help guide me through this “second iteration.”

Even though my code was deleted and caused me a lot of stress I would like to take a few positives out of it. The first positive was being able to better prompt the AI on the steps that were harder for it to get to work on the first iteration. The second positive was being able to see the difference between the two times it happened. Even though I can't compare the code exactly between the two I can say that the AI was much more efficient and better the second time through.

Conclusion

When it comes to AI and its use of its knowledge design patterns, it is significantly larger than the knowledge that I have. The AI accesses a large database of information that contains all that information at the drop of a hat. It is able to understand it, use it, and analyze it. It is trained off of these design patterns. If I want to know about these design patterns I have to go online and find a resource that details that kind of information to me.

In terms of mimicry, all AI knows is essentially mimicry, patterns, and algorithms. AI is not an independent free thinker, it needs to have seen the information somewhere before in order to do something with it. It is not capable of thinking of something abstract or random to say. All the code that an AI knows comes from some human code once before, it was trained on human code. It goes back to my previous statement: AI needs to have seen something before in order to do something with it.